

PAPER-FIRST AGENT LOOPS

Sloop

Choose sloop, not slop.

A local-first meta-IDE for defining, running, and supervising nested coding-agent workflows as Markdown, evals, diffs, and Git history.



Agent work disappears into chat.

Prompt soup

Intent, constraints, decisions, and caveats are scattered across a session transcript.

Mystery diffs

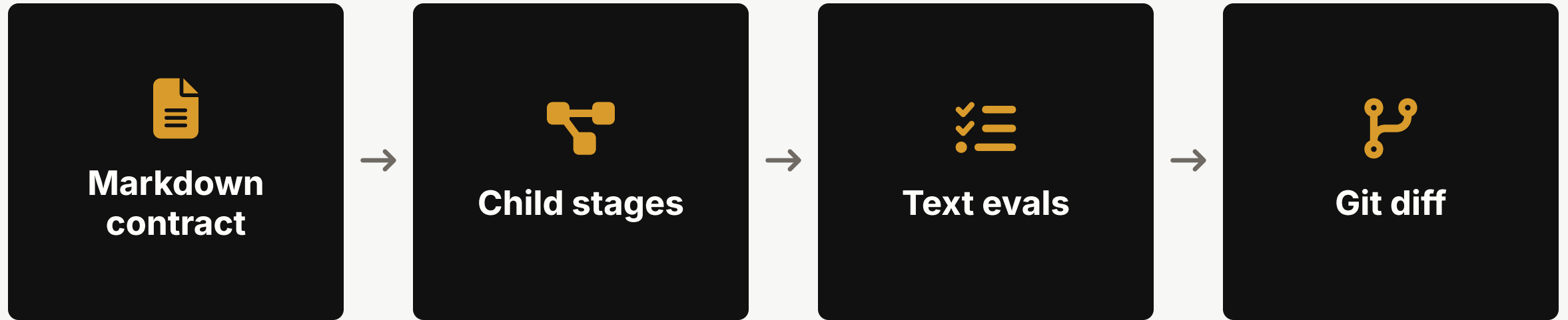
Code changes land without a durable contract that says why they exist or how to check them.

Manual babysitting

When requirements change, humans chase the cascade across docs, plans, tests, and code.

Today's agents can produce a lot. Sloop asks a better question: what keeps that work inspectable, rerunnable, and correct?

Make the loop the primitive.



Instead of one-off agent sessions, Sloop turns software work into explicit documents that define what should happen, what may change, and what must pass before changes are accepted.

A loop doc is both paper and control plane.

loops/product-requirements.md

```
status: running
agent: pi
autoApply: true
children: architecture, build
```

Product Requirements

The product goal, constraints, acceptance criteria, and downstream stages live in one readable Markdown source of truth.

Human-editable

Users can pause, edit, resume, and let the cascade continue from the visible document.

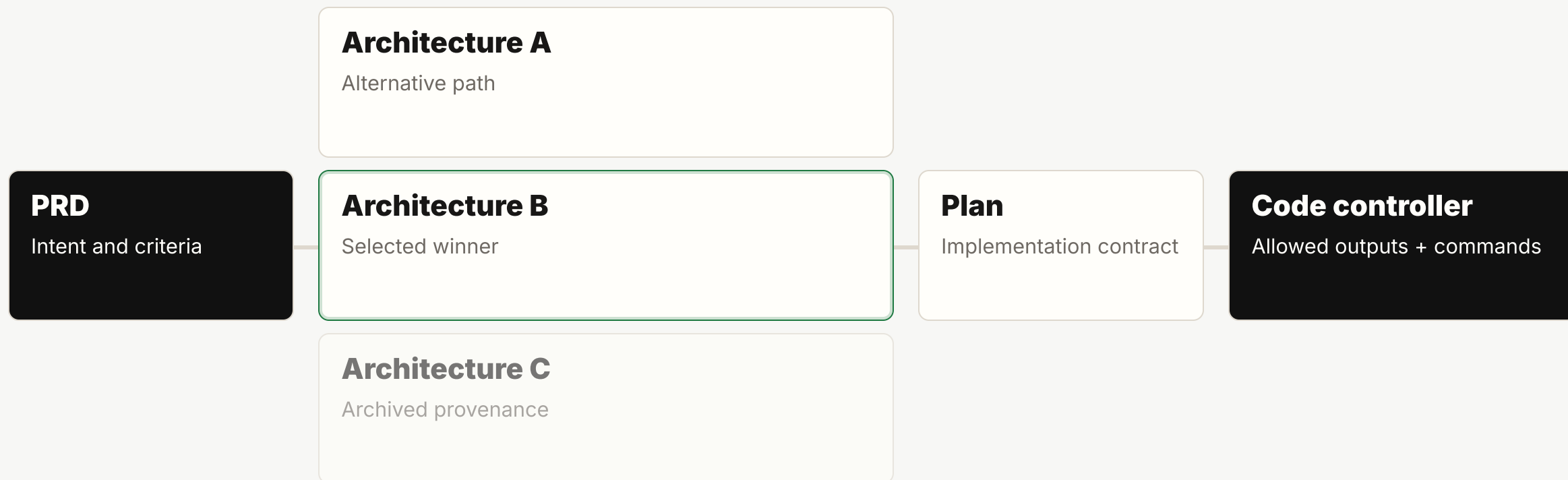
Evaluated

Text criteria become deterministic checks, commands, schemas, fixtures, or review prompts.

Auditable

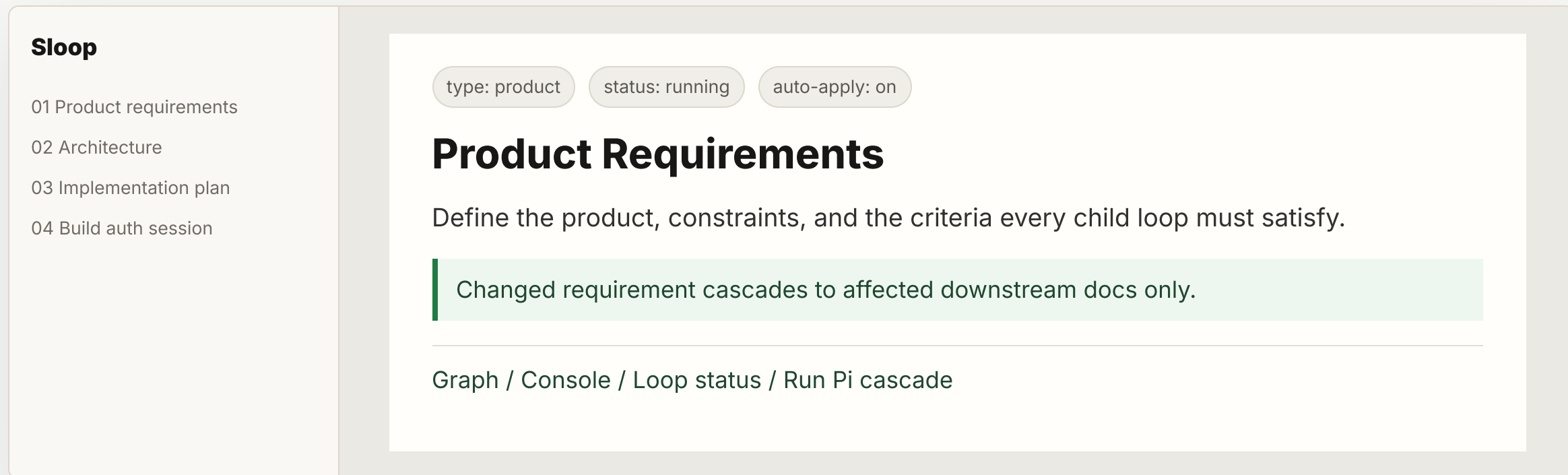
Every run, diff, pause, failure, retry, and rollback point is backed by Git history.

Users define the pipeline. Sloop runs the cascade.



The PRD to architecture to plan to build flow is just a template. Sloop's real move is letting teams invent their own nested agent workflows.

A document surface, not a command center.



The default mode stays calm: frontmatter as controls, diffs near the changed text, loop state at the bottom, history available when needed.

Code stages are controlled by Markdown.

Parent loop

Owns intent and inherited eval criteria.



Controller doc

Owns outputs, commands, retries, and failure evidence.



Pi attempt

Edits only affected docs and allowed paths.

loops/build/build-auth-session.md

```
outputs:  
- src/auth/**  
- tests/auth/**  
commands:  
- npm test -- auth  
maxAttempts: 3
```

If eval fails, evidence returns to the same loop and Pi retries with the visible workspace state.

Cascades are diff-driven, not hand-waved.

01 User edits Markdown

Requirements, evals, stages, or controller docs change.

02 Sloop reads the Git diff

The changed intent becomes the impact boundary.

03 Affected loops run

Downstream docs and code controllers update only where needed.

04 Evals decide

Passing changes apply; failing ones remain visible with evidence.

Git is not just storage. It is the audit trail, rollback system, review surface, and cascade sensor.

WHAT WE ARE BUILDING



Define the loop. Trust the work.

- ✓ Workflows defined as Markdown
- ✓ Runs gated by evals and commands
- ✓ Changes reviewed through Git diffs

Choose sloop, not slop.